TTB · CERTIFICATE OF LABEL APPROVAL

AI-Powered Alcohol Label Verification

Verifying that a label matches its application — in seconds, at agency scale.

Approach & Design Alex · June 2026

READ THIS FIRST

A prototype now, an in-boundary system later

The demo stack and the production stack are deliberately different, because the agency's firewall and FedRAMP boundary rule out public cloud services in production.

PROTOTYPE · THE TAKE-HOME

Modern indie stack

- Warm container on Railway / Render / Fly
- Supabase (Postgres + pgvector) for the full demo
- Anthropic API for the model
- Shared-passcode gate, nothing sensitive stored

PRODUCTION · IN THE AGENCY

In-boundary on Azure (FedRAMP)

- App Service / AKS · Azure Database for PostgreSQL
- Azure Blob for label images
- In-boundary model, no public model APIs
- Self-hosted observability, no external SaaS

The firewall that killed the prior vendor is why production can't use Vercel, Supabase, or Railway. The architecture isolates the model, datastore, and observability seams so the swap is cheap.

THE PROBLEM

Skilled agents, buried in routine matching

150,000

COLA applications reviewed per year

47

agents handling the entire queue

~5 sec

the speed bar — or the tool goes unused

Verification is manual. An agent eyeballs the label artwork against the typed application, field by field. Roughly half the day is routine matching, not judgment — and the prior scanning vendor was abandoned at 30 to 40 seconds per label.

DISCOVERY

What the people doing the work told us

Sarah · Deputy Director

Under ~5 seconds or agents abandon it. Peak-season batches of 200–300 are a real bottleneck.

Marcus · IT

Standalone proof-of-concept, no COLA integration. The firewall blocks external ML endpoints.

Dave · Senior agent (28 yrs)

Judgment is real. 'STONE'S THROW' and 'Stone's Throw' are the same product.

Jenny · Junior agent

The government warning must be exact — caps and bold. Photos are often imperfect.

THE REFRAME

Remove the grind, not the decision

TODAY

Verification and approval are fused

- Check every field by eye, every time
- Half the day on routine matching
- No time for the cases that need judgment

WITH THE TOOL

The tool matches; the agent judges

- Auto-verify every application at intake
- Triage by confidence into three lanes
- Agent resolves the exceptions, owns the call

The human keeps the decision and the legal accountability.

SOLUTION

One queue, three lanes

Every application is verified the moment it arrives and sorted by confidence. The agent works a pre-sorted queue.

✓ MATCH

High confidence

Every field agrees. Bulk-approve in one click.

× MISMATCH

Clear problem

A field disagrees — flagged with the exact field.

? REVIEW

Low confidence

Unreadable or a near-miss. Sent to a human.

The agent spends time only in the second and third lanes — that is the time saved.

THE PIPELINE

Submissions in, decisions back to the record

INTAKE · VIA COLAs ONLINE (~150,000 / YR)

Wine
COLA + label art

Distilled Spirits
COLA + label art

Malt Beverage
COLA + label art

Imports
COLA + label art

LabelCheck
verify · triage

LabelCheck sits inside the existing pipeline: it never replaces COLAs Online or contacts applicants.

BACK TO THE TTB SYSTEM OF RECORD

DISPOSITION
- Approve → COLA issued
- Return for correction

WRITTEN BACK TO COLAs ONLINE
- Decision recorded
- Applicant notified by email

ENDING
- Published to Public COLA Registry
- Auto-reject after 30 days

HOW IT WORKS

The model reads. The code decides.

1 **The model only transcribes**

It reads the label text from every face; it renders no verdicts.

2 **The code does the matching**

Per-field rules — brand & type fuzzy, ABV & contents normalized — in inspectable, testable code.

3 **Confidence is computed, not claimed**

Derived from match margin and legibility — never the model’s self-reported number.

THE HARDEST CHECK

Government Warning

- Present on any face
- Verbatim wording
- ‘GOVERNMENT WARNING:’ in ALL CAPS — strict
- Bold is best-effort → routed to a human

END TO END

The application lifecycle

1

Submitted

via COLAs Online

2

Auto-verified

AI reads label vs form

3

Triaged

into three lanes

- match
- mismatch
- review

4

Who handles it

by lane

- Supervisor · matches
- Specialist · exceptions

5

Decision

on each exception

- Approve → COLA
- Return for correction

6

Ending

resolved

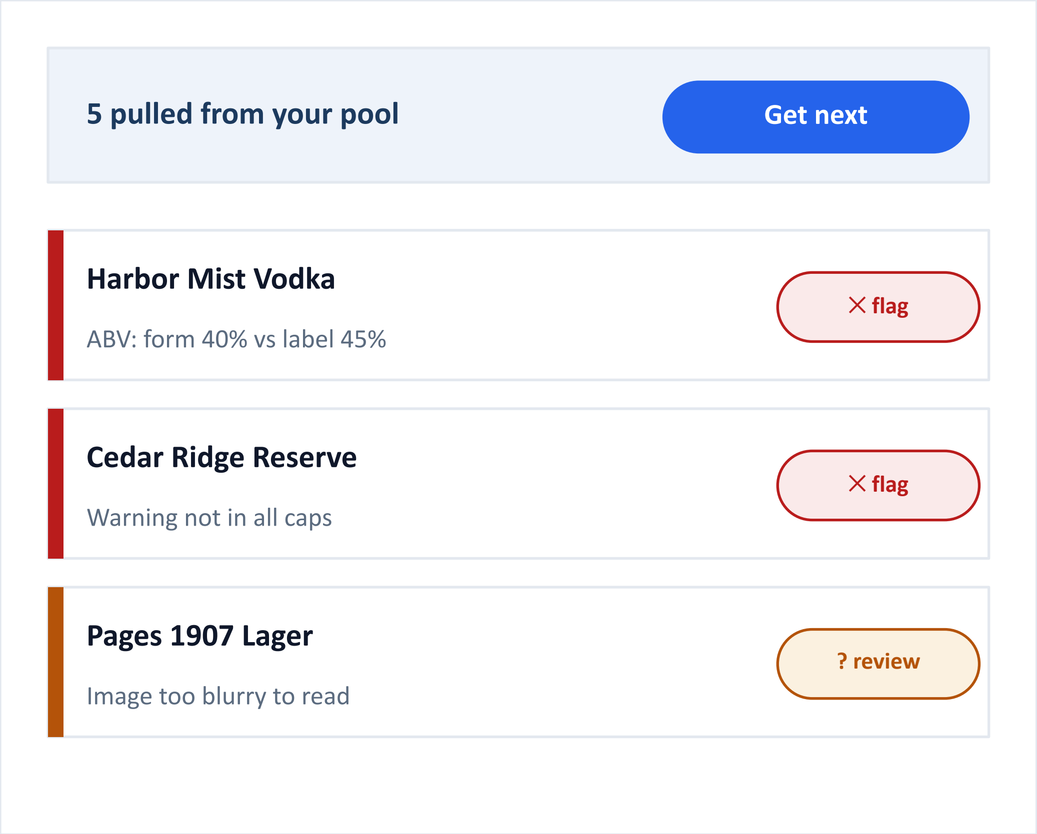
- COLA issued
- Auto-rejected ≥30d

A supervisor clears the clean majority in bulk; specialist agents handle only their beverage type's exceptions. Rejection is automatic when a 30-day correction window lapses — there is no manual reject.

Like a smart inbox

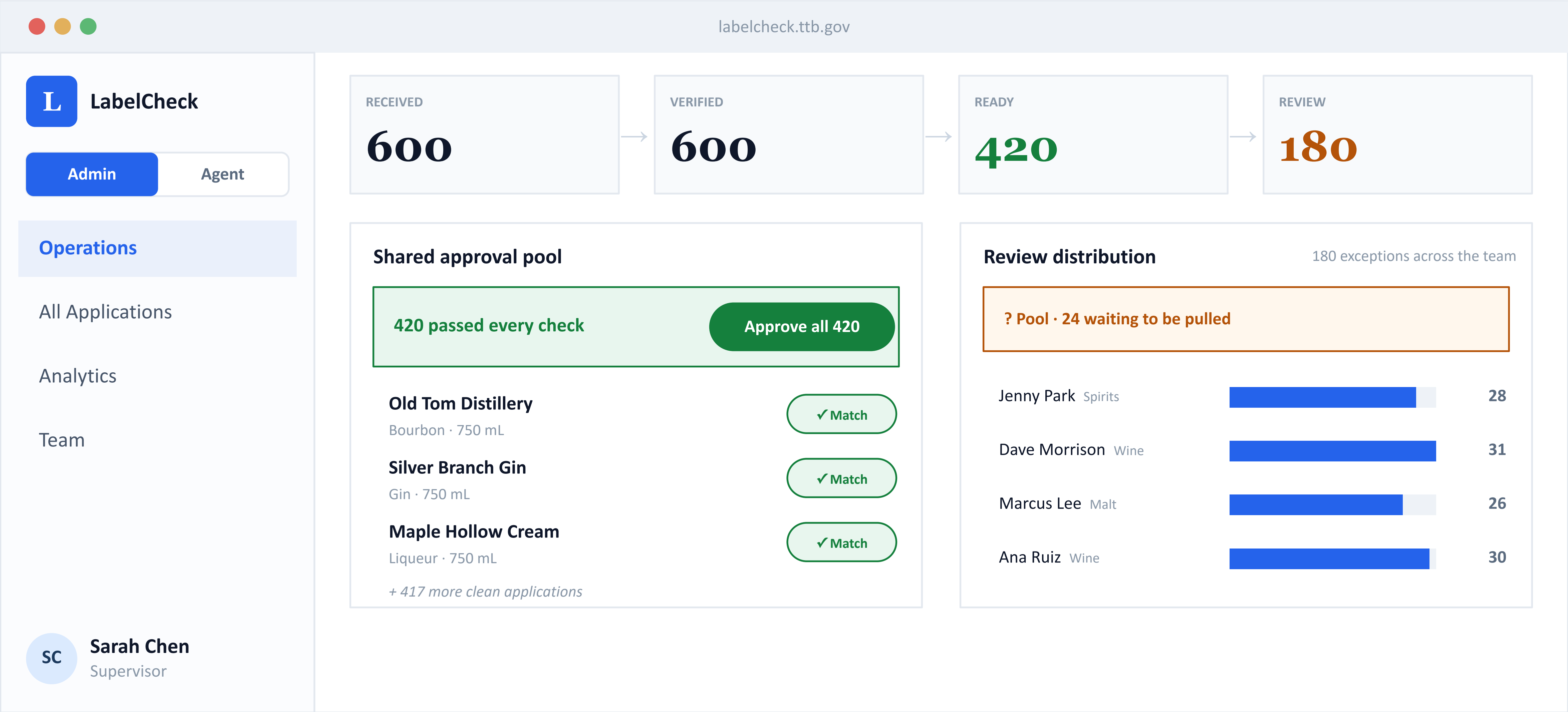
“5 exceptions in your queue, pulled from your specialty's pool.”
You only see the cases that need a human.

- Pull the next exception from your pool
- Application vs label, side by side
- Decide: Approve, or Return for correction
- Accessible: colour + icon + text, large targets



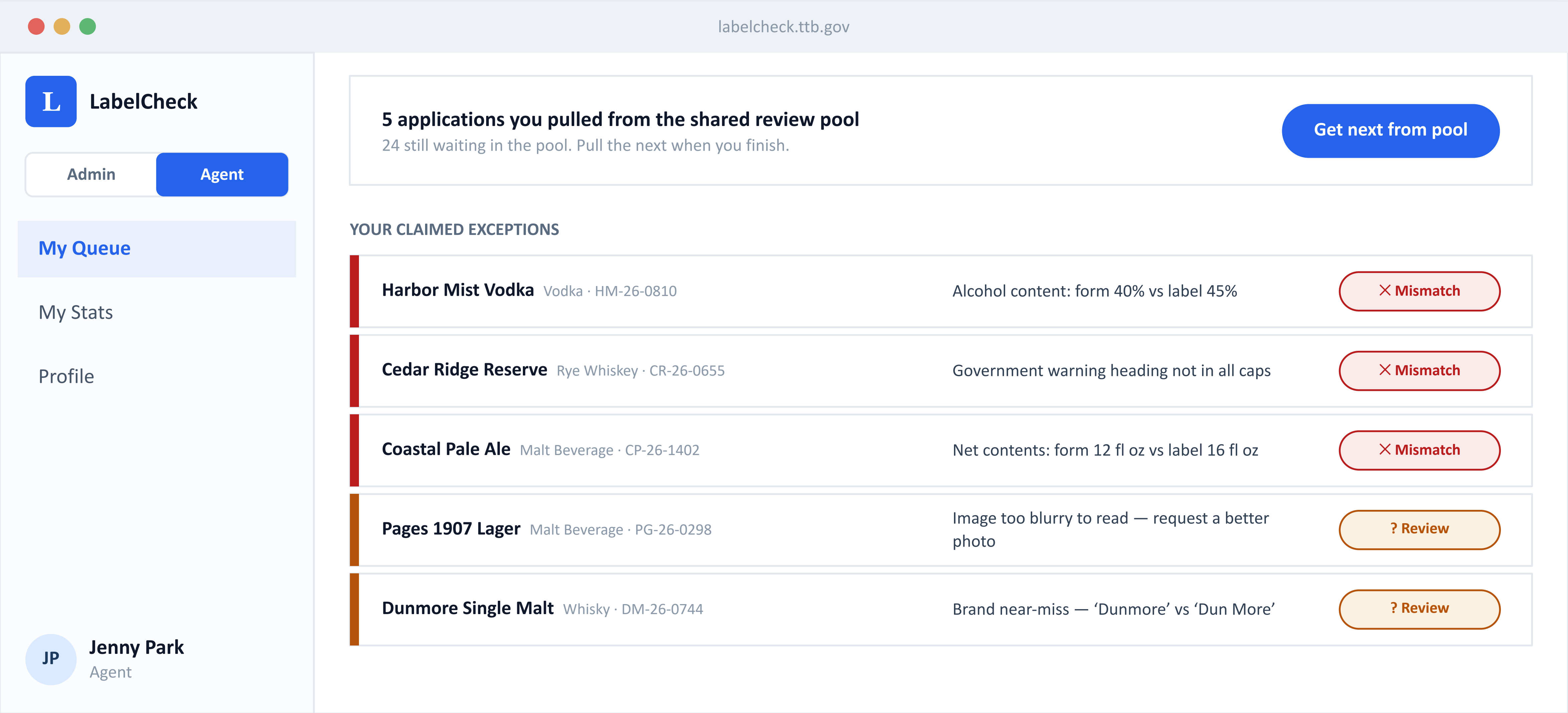
PRODUCT · ADMIN VIEW

The Operations dashboard



PRODUCT · AGENT VIEW

An agent's queue — pulled from the pool



BUILT TO SCALE

Specialists, an assistant, a managed knowledge base

Beverage specialists

Exceptions route to the agent who specializes in that label type — wine, distilled spirits, or malt beverage — with overflow so nothing stalls. Specialists read their own type faster and more accurately.

Read-only assistant

A bottom-right helper that answers questions, onboards new agents, and summarizes each user's own role-scoped numbers. It explains and summarizes; it never decides or changes records.

Managed knowledge base

Admins upload the documents the assistant is grounded in — chunked, embedded, and indexed — so it can cite only approved sources. Observability measures whether answers stay grounded.

Each builds on the same access scoping and the same self-hostable, in-boundary AI stack.

HOW IT FITS THE TTB PROCESS

Where applications come from, where decisions go

Applicant
submits via COLAs Online

→

COLAs Online
system of record

→

LabelCheck
verify · triage · agent decides

→

COLAs Online
notifies applicant · publishes

In-boundary by design. Our tool reads applications from and writes dispositions back to COLAs Online, all inside the agency's Azure boundary — never the public internet. **It sends no email: COLAs Online owns all applicant notification and Public Registry publishing.**

ARCHITECTURE & STACK

Deliberately boring, appropriately scoped

Ingestion → Extraction → Matching → Triage

Vision model behind a swappable adapter · mock adapter for offline dev & tests · a thin work router for the exception pool

TypeScript end-to-end

One language, one repo — Next.js front and back

Claude Sonnet 4.6 (default)

Accuracy-safe, swappable by config; one call per application

Postgres + JSONB (production)

Prototype persists nothing; one always-warm container

Reading the label, inside the boundary

The model only reads the label; code does the comparison. So it is a benchmarked, swappable component, and in production it never leaves the agency's network.

RECOMMENDED · IN AZURE GOV

Azure OpenAI vision

- FedRAMP High in Azure Government (DoD IL4 / IL5)
- US vendor, inside the agency's boundary
- No external endpoint; the prior vendor failure, solved

AIR-GAPPED · SELF-HOSTED

Open OCR-VL on agency GPUs

- olmOCR (US, Apache, fully auditable) for provenance
- GLM-OCR / Qwen-VL for top accuracy, security-reviewed
- Small models (under 1B to 7B), cheap to run in-house

The top open OCR models are mostly non-US in origin, a procurement flag for Treasury. Whichever path, the model is chosen by a bake-off on real TTB labels, not a public benchmark.

Built for accountability, evaluated continuously

Two things a regulatory AI system must get right: an unalterable record of what was decided, and proof that the AI is actually accurate. Both run in-boundary.

THE DATA MODEL

PostgreSQL, accountable by design

- PostgreSQL + JSONB; images in object storage
- Append-only disposition & audit, the unalterable record
- Prototype stores nothing (PII); knowledge base on pgvector

EVALS & OBSERVABILITY

Measured, not assumed

- OpenTelemetry traces, self-hosted Langfuse / Phoenix
- Golden set from the Public COLA Registry; false-negative rate is key
- Agent corrections = free ground truth; guardrails enforced

The regulatory record can't be rewritten; the AI's quality is checked against a golden set and the agents' own corrections, on a self-hostable, in-boundary stack.

Proving it finds real errors

How the tool earns trust before it is relied on: real applications it should clear, and planted defects it must catch — every case scored against a known answer.

PASS SET · REAL DATA

Approved pairs from the registry

- Pulled from TTB's Public COLA Registry (free, 1999–present)
- Each case = real form fields + the real label image
- Should all clear → measures false positives (crying wolf)

FAIL SET · MANUFACTURED

Known, planted defects

- Change a form field, or AI-generate a defective label
- Every defect recorded as the known correct answer
- Should all be caught → measures false negatives

Scored against known answers

Lane accuracy · per-field precision / recall · warning-check accuracy · latency p95. **False-negative rate is the headline — a bad label cleared is the costly error.**

Demo batch: ~150 real approved + ~50 planted defects in one run → grouped by lane, then a scorecard ('caught 48 of 50; here are the two it missed and why'). The registry holds only approved forms, so failing cases are manufactured; agent corrections add fresh labeled cases over time.

CONSTRAINTS & REQUIREMENTS

The box we built within

5-second latency

the hard acceptance bar

Accessibility

WCAG AA, older users

In-boundary model

firewall blocks external

No PII stored

ephemeral by design

Requirements as testable acceptance criteria

Clean registry pair → all green · ABV mismatch → flagged on ABV · title-case warning → flagged
'STONE'S THROW' vs 'Stone's Throw' → match · blurry image → needs-a-better-image, not an error

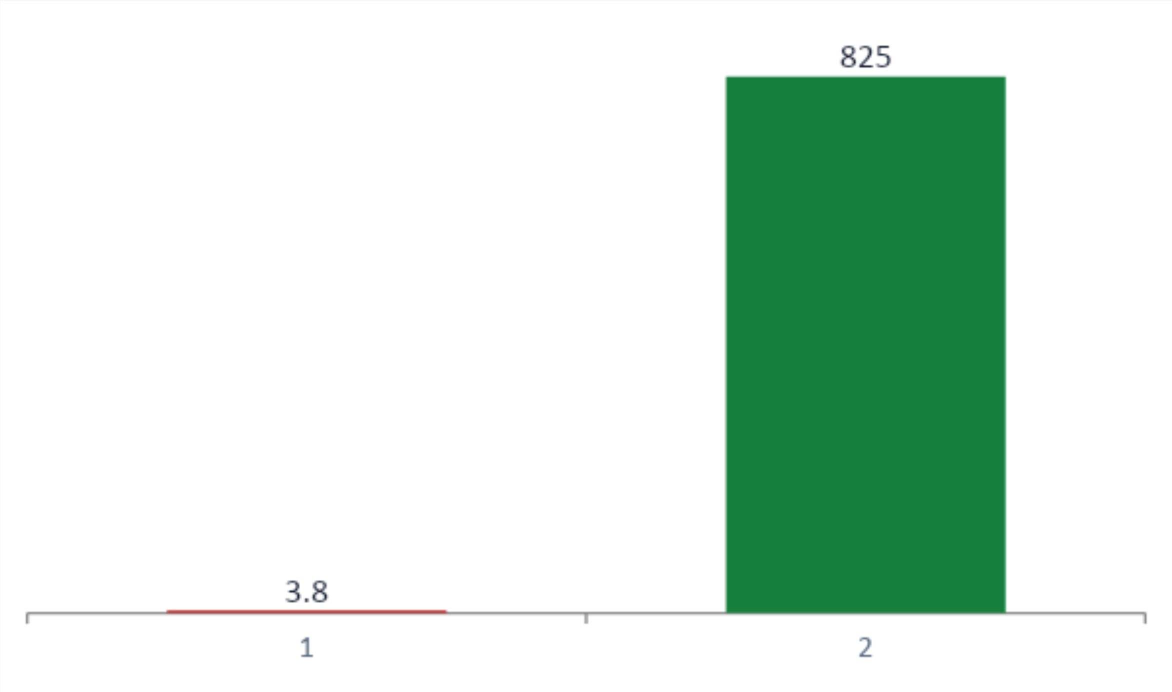
KEY DESIGN DECISIONS

Every one a real trade-off

Decision	Why	Trade-off accepted
Model reads, code decides	Judgment stays testable & tunable	More glue code than letting the model judge
Confidence computed in code	LLM self-reported confidence is miscalibrated	Derive it from match margin + legibility
Full-resolution images	Protect the tiny government-warning text	More tokens & latency than downscaling
Pull-based exception routing	Self-balances; only exceptions are routed	No specialized push assignment by default
No persistence (prototype)	PII; IT required nothing stored	Batch state lost on restart

Recorded as ADRs in the design docs.

It pays for itself, many times over



$ thousands per year

~$3,800 / yr

Inference at full national volume (default model)

$690k–$960k / yr

Agent labour saved — about 7 agents' capacity freed

< 1%

Operating cost as a share of the benefit

Realized savings depend on adoption, accuracy, and the agency's auto-clear policy.

SCOPE & ROADMAP

Honest boundaries, a credible path

NOW · Prototype

- Verify flow + three-lane triage
- Operations, agent & admin dashboards
- Specialists, assistant & knowledge base
- Standalone — nothing stored

NEXT

- Batch handling at peak season
- Imperfect-image robustness
- Latency hardening to <5s
- Evals & observability harness

LATER · Production

- In-boundary model on Azure
- COLA ingestion + write-back
- Auth, roles, audit, datastore
- Self-hosted observability + analytics

IN ONE SENTENCE

A second set of eyes that does the boring matching in seconds — so agents focus on judgment.

Build first: the core single-application verify flow — then batch and imperfect-image handling.